

CS 133 Lab 1 Report: High-Performance GEMM

Name: Alexander Neary

UID: 805935345

Date: April 2026

1. Optimization Description

1.1 GemmParallel

The sequential implementation from ``gemm.cpp`` used the standard i, j, k loop order. Since we know that a CPU will access memory in chunk sizes, I drew out the process order for matrix multiplication. Since matrices are stored as 2d arrays, each element of the array is a row (with each element in that row being the column).

Due to k being the innermost loop, matrix B is iterated by row first, then column. Matrix A on the other hand is iterated by column first and then row, meaning it can grab a row from cache and run operations on that singular row for multiple loops of k . We want to do the same thing for B.

Optimizations Performed:

- **Loop Order:** Changed from i, j, k to i, k, j .
- **Parallelization:** Added ``#pragma omp parallel for`` to the outermost loop i .
- **Reasoning:** In the i, k, j order, both matrix A and matrix B are iterated by column first, so the cache locality is maximized during matrix multiplication. Parallelizing the i loop allowed for each row of the output matrix C to be computed in parallel.
- **Performance Increase:** (On my local machine, 2048 x 2048 x 2048)
Sequential GEMM: 0.383 GFlops
vs
Parallel GEMM: ~170 GFlops

1.2 GemmParallelBlocked

The problem with the non-blocked parallel version from ``omp.cpp`` is that for each row of matrix C that is computed, the processor has to read the entirety of matrix B. Once the outer loop i is iterated, all of matrix B will need to be read again. This means that each thread is reading the entirety of matrix B within its loops. Instead of using 1 row of A and all of B per thread, we can use a small block of A and a small block of B to compute the partial matrix product for that block in C. This will better utilize the cache (memory locality) by avoiding fetching the entire matrix B per thread.

- **Block Size chosen:** $B = [128]$

- **Reasoning:** Intel Xeon Platinum 8175M 24-core CPU (a custom model for Amazon AWS), each thread has its own L1d 32KiB Cache. 32KiB = 32768 bytes → since a block of A and a block of B is needed, calculate half of the space 32768 bytes / 2 = 16384 bytes → 16384 bytes of space fits how many words (a word is 32 bits or 4 bytes): 16384 bytes / 4 bytes = 4096 words → 4096 words is a 64 x 64 block of words. Or at least that's what I thought, because when I tested with a block size of 128 it significantly improved performance.

- **Implementation:** Block size (and number of blocks) is stored for calculations before any loops or parallelization. The first two loops are for iterating through result matrix C, and it's parallelized using `#pragma omp parallel for collapse(2) schedule(static) num_threads(8)`.

- Spawns a team of threads, dividing iterations of the for loops (plural because collapse(2) splits up each sub-block of C into a thread instead of simply splitting up by row).
- Static scheduling used to divide equal-sized blocks to threads at compile time, has minimal overhead.
- 8 Threads because an m5.2xlarge instance uses 8 vCPUs (4 physical cores with hyperthreading to get 8 threads).

The following loop is used to iterate over A and B (by block), giving the block offset. The final 3 inner loops are the i, k, j ordered matrix multiplication from `omp.cpp`, adjusted for the tiling format. I also used `#pragma omp simd` to improve performance.

- Allows each thread to perform calculations on a bigger scale.

Other improvements:

- Row pointers to utilize local memory more efficiently.
- Loop unrolling to do more calculations per memory access.

- **Performance Increase:** (Local Machine, 2048 x 2048 x 2048)

Parallel GEMM: ~170 GFlops

vs

Parallel-Blocked GEMM: ~410 GFlops

- **Increase per optimization:** (AWS m5.2xlarge Instance, 4096 x 4096 x 4096)

- *Tiling/Blocking (Block Size 64):* ~35 --> ~60 GFlops (utilizing loop permutation $[i, k, j]$ order)
- *Value Hoisting:* ~60 --> ~65 GFlops
- *SIMD Vectorization* (`#pragma omp simd`): ~65 --> ~70 GFlops
- *Loop Unrolling (2 Way):* ~70 --> ~80 GFlops
- *Local Memory (Row Pointers):* ~80 --> ~87 GFlops
- *Block Size Update (128 instead of 64):* ~87 --> ~95 GFlops

- **Scalability w/ Different Threads:** As mentioned above, the m5.2xlarge instance has 8 vCPUs which equates to 4 physical cores with hyperthreading (essentially 8 virtual cores/threads).

- `num_threads(1):` ~14 GFlops
- `num_threads(2):` ~35 GFlops
- `num_threads(4):` ~50 GFlops

- *num_threads(6)*: ~65 GFlops
 - *num_threads(8)*: ~90 GFlops
 - *num_threads(10)*: ~60 GFlops
-

2. Performance Evaluation

The following results were obtained on the **m5.2xlarge** AWS instance with a problem size of 4096^3 .

Version	GFlops	Time (s)	Execution Mode
Sequential (Baseline)	0.2441	562.924	Single Thread
GemmParallel	36.8627	3.7284	OpenMP + Multi-thread
GemmParallelBlocked	99.5091	1.38117	OpenMP + Tiling

Performance Range achieved: A

3. Analysis and Observations

- **Scaling:** Adding parallelism to the sequential version of GEMM allowed for multiple cores to calculate the result at the same time, combining the overall result to receive matrix C. This change provided a major performance improvement.
- **Cache Impact:** Through implementing blocking, the GFlops even on just my local machine jumped up by over 100. Splitting up C by blocks, having each thread calculate the result for that block of C, and grabbing blocks of A and B from memory instead of full rows/columns at a time all increased the performance significantly. These changes maximized cache locality between cores, providing a worthwhile speedup.
- **Challenges:** Trying to wrap my head around the full process for blocked parallel matrix multiplication was difficult. I ended up making a couple different drawings to visually grasp the order of operations/loops, as well as where each variable ends up after a calculation. Once I broke the problem down, I was able to work through it sequentially to best optimize performance. I did end up getting stuck at about 70 GFlops before increasing the block size and doing some loop unrolling, which got me to the A performance range.